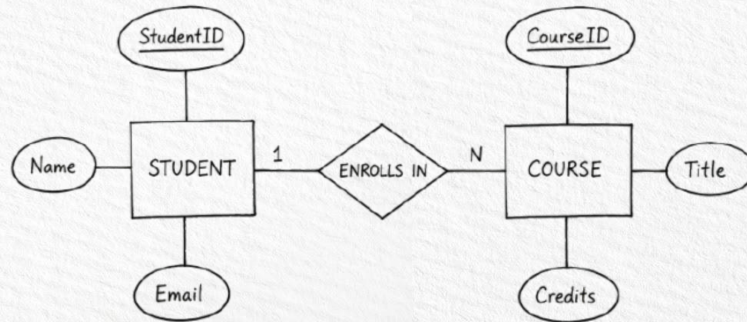


CSC343 Lecture 2:

Intro to DDL and ERDs continued



Today's Agenda

- Admin Notes
- Last Week's Review (Active)
- ERD Multiplicity (Lecture)
- ERD Multiplicity Activity (Active)
- Intro to DDL (lots of lecture)
 - Creating tables
 - Defining attributes
 - Creating (some) constraints
- DDL activity (Active)

Admin Note: Anonymous Feedback

"I hope that new terminologies on the slides will be written in full for the first time. Many of the new terms like ERN or DBMS were not, which is a bit hard for me to grasp the context and later when I review the lecture slides.

Also, the transition to SQL code examples was bit abrupt when the class hasn't been taught SQL yet, making it a bit hard to process when I was trying to understand weak entities and how the SQL examples relate at the same time. I think it would have been easier if you could label the code a little bit so we can better benefit from the example."

This is super helpful, thank you!

Admin Note 2: Quercus!

Quercus is ... gone. Don't log in!

We'll use: <https://naazsibia.github.io/csc343hy5-2026/>

Simpler Link: tiny.cc/csc343h5y

Recap Quiz!

The winner gets a \$5 gift card.

If there are multiple winners, we'll have a draw!
(Because Naaz doesn't want to go broke!)



Which of the following is **not** a benefit of a Database Management System (DBMS)?

- Enforces data integrity constraints automatically
- Eliminates the need to think about how data is structured
- Controls concurrent access by multiple users
- Provides a standard way to query and manipulate data

Which of the following is **not** a benefit of a Database Management System (DBMS)?

- Enforces data integrity constraints automatically
- ~~Eliminates the need to think about how data is structured~~
- Controls concurrent access by multiple users
- Provides a standard way to query and manipulate data

A DBMS is *more* design thinking, not less.



Fun fact: The first commercial database management system was created in the 1960s for an Apollo moon mission project by IBM and North American Aviation!

A **weak** entity

- Has fewer attributes than a strong entity
- Has a primary key that is always shared with another entity
- Only appears on the N-side of a 1:N relationship
- Cannot exist without its identifying relationship to another entity

A **weak** entity

- ~~Has fewer attributes than a strong entity~~
- ~~Has a primary key that is always shared with another entity~~
- ~~Only appears on the N side of a 1:N relationship~~
- Cannot exist without its identifying relationship to another entity

Weak = existence-dependent. Its partial key only makes sense relative to the owner.



In the relational model, a **key** is best described as...

- Any attribute that appears in more than one table
- The first column of a relation, by convention
- An attribute or set of attributes that uniquely identifies a tuple
- Any attribute declared with the UNIQUE constraint

In the relational model, a **key** is best described as...

- ~~Any attribute that appears in more than one table~~
- ~~The first column of a relation, by convention~~
- An attribute or set of attributes that uniquely identifies a tuple
- ~~Any attribute declared with the UNIQUE constraint~~

Weak = existence-dependent. Its partial key only makes sense relative to the owner.



Fun fact: Social Security Numbers became a popular database key even though they were created for tracking earnings, not identifying everyone everywhere. Don't just make something a key because it is unique!

Principles for a good key:

Principle	Meaning	Example
Unique	No two rows share it	StudentID, OrderID
Stable	It does not change often	Not phone number or address
Minimal	No extra attributes needed	Use StudentID, not StudentID + Name
Not sensitive	Safe to reference in other tables	Avoid SSN as a public database key
Required	Every row must have one	No blanks or missing values

In a Chen ERD, a *relationship* set is drawn as...

- A rectangle
- An ellipse
- A diamond
- A double rectangle

In a Chen ERD, a *relationship* set is drawn as...

- A rectangle
- ~~An ellipse~~
- ~~A diamond~~
- ~~A double rectangle~~

Entity Relationship Diagrams (ERD)

Recap

- Represent the relational model
 - **Entities** - like objects, things that are different from each other
 - Have **keys** to uniquely identify them
 - Have attributes
 - Types: strong entity, weak entity

Entity Relationship Diagrams (ERD)

Recap

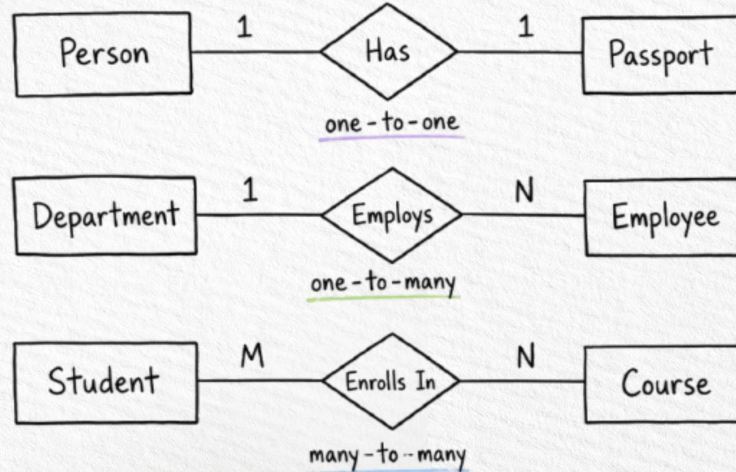
- Represent the relational model
 - **Relations** - represent relationships between objects
 - Can have their own attributes
 - Typically identified by the entities they relate
 - Types: non-identifying and identifying relationships, binary, tertiary, n-ary relationships
 - Participation: total vs. partial

A simple wooden frame with a light brown, grainy texture surrounds a white rectangular area. The frame is composed of four pieces meeting at the corners.

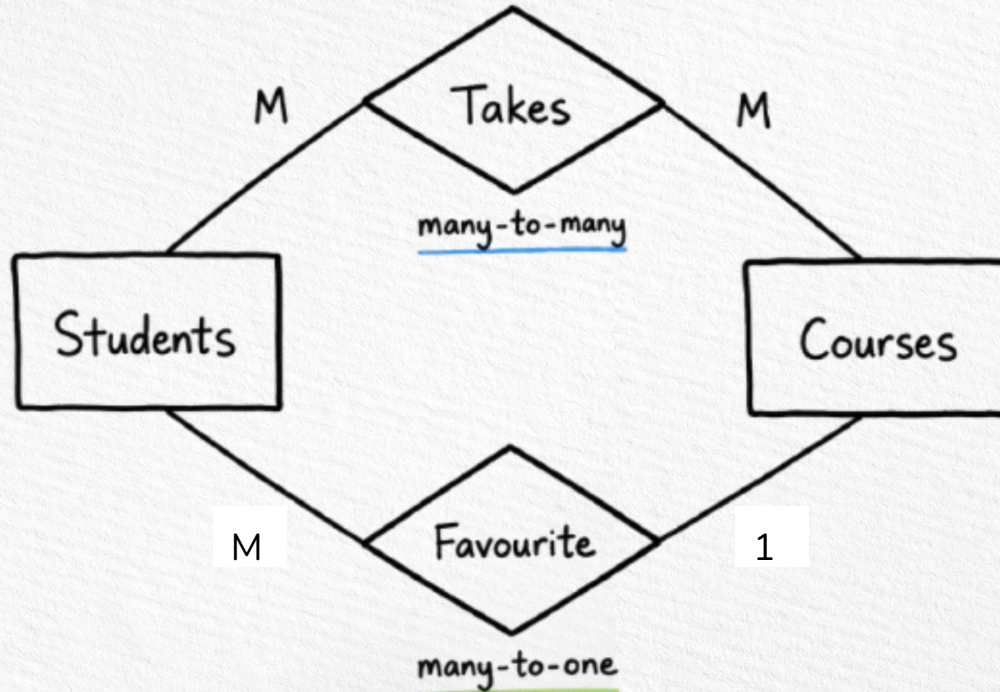
New Content!

Multiplicity

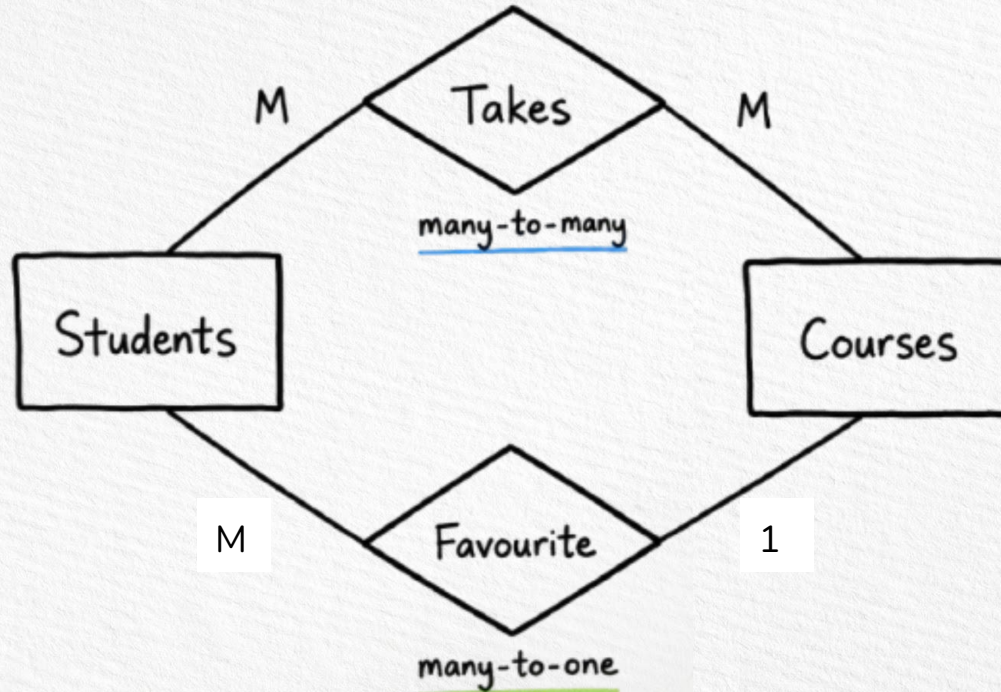
- **Multiplicity** is the *cardinality* (maximum number of relationships that occur) and the *participation* (in a relationship)



Example: Many-One Relationship



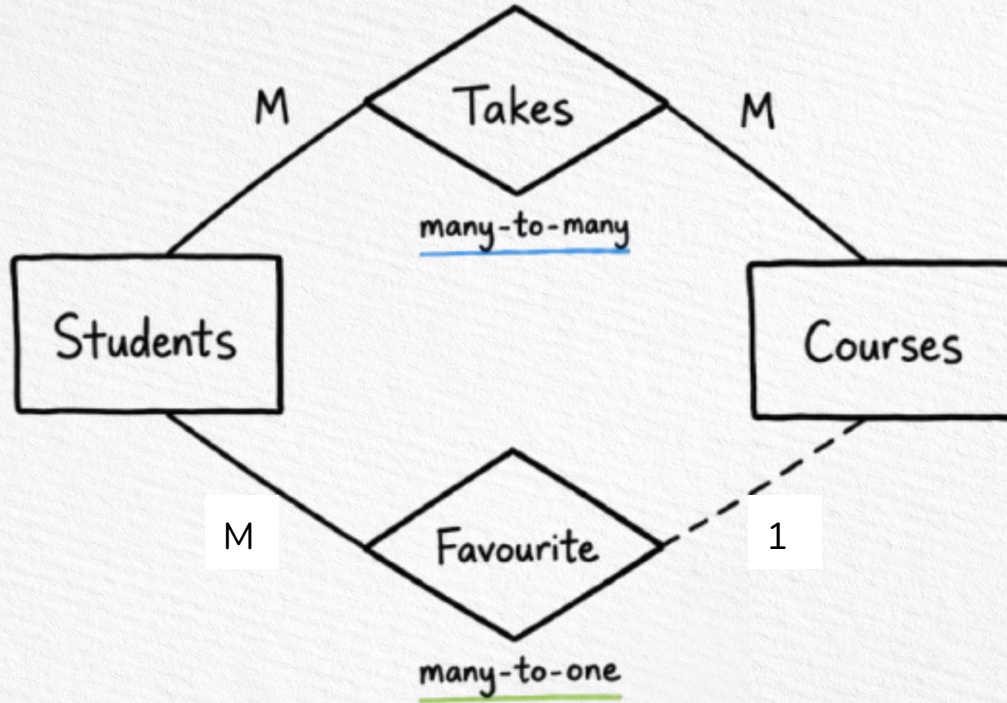
Example: Many-One Relationship



Note: two relationships connect the same entity sets, but are different!



Example: Many-One Relationship



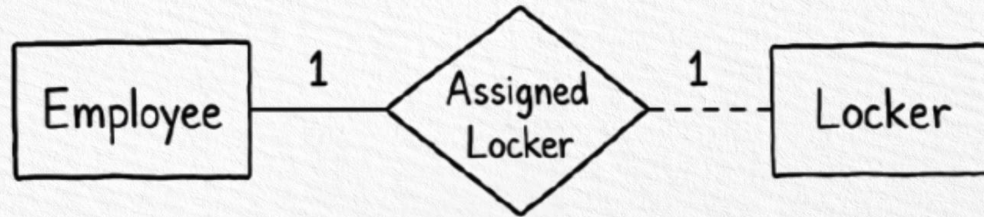
Note: can have partial participation!



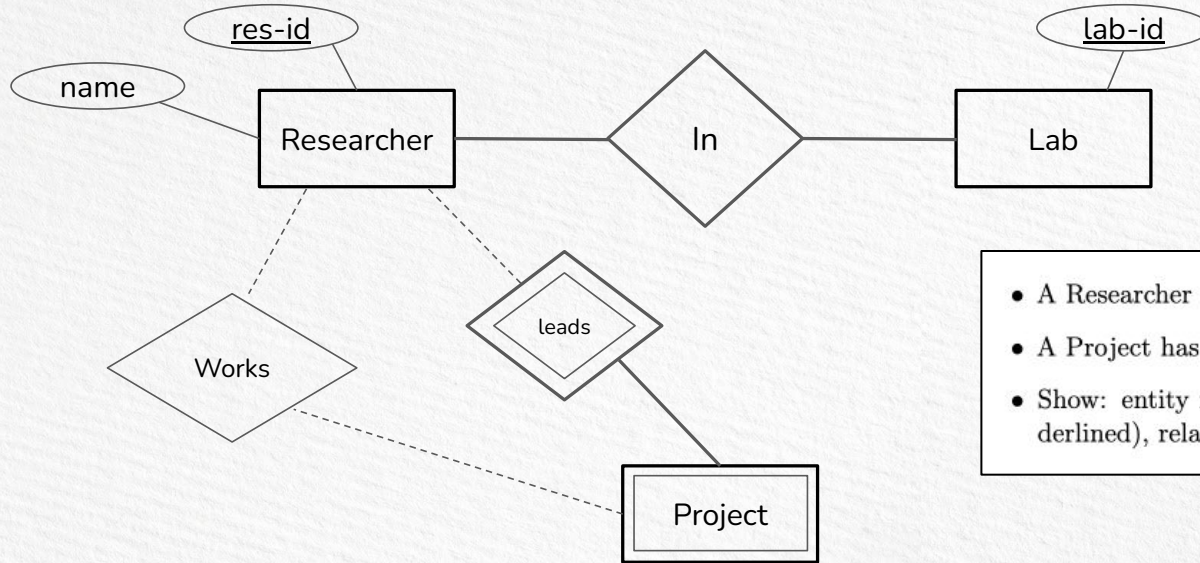
Example: One-One Relationship

Consider **Assigned Locker**:

- Some lockers don't have employees assigned
- But every employee must have a locker



Exercise: edit last week's ERD to include multiplicity!

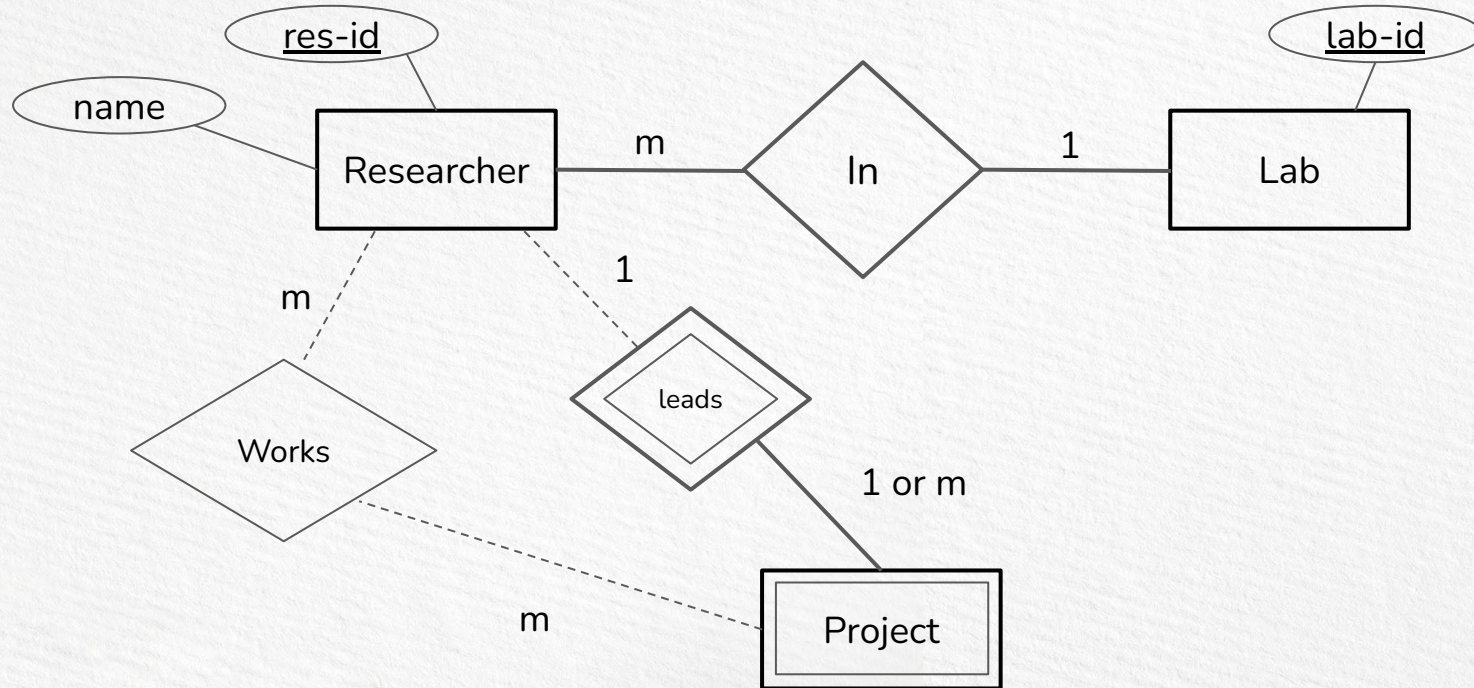


A researcher works in one lab. A lab can have many researchers.

- A Researcher works on **many** Projects.
- A Project has **exactly one** Lead Researcher.
- Show: entity rectangle, at least two attributes (one key, underlined), relationship diamond, cardinality.

Note: I changed some names to make them fit!

Exercise: edit last week's ERD to include multiplicity!



Break!

Database Schemas in SQL

- SQL is primarily a *query* language
 - **DML:** Data Manipulation Language
- But it also includes a *data-definition* component to describe database schemas!
 - **DDL:** Data Definition Language

Creating a schema

- In Postgres, every database can have multiple schemas. The default schema is public
- To create a new schema:

```
CREATE SCHEMA myschema;
```

- To switch to an existing schema:

```
SET search_path TO myschema;
```

Creating (Declaring) a Relation

- Simply

```
CREATE TABLE <table_name> (  
    <list of elements>  
);
```


Deleting a Relation

- Simply

```
DROP TABLE <table_name>;
```

Elements of Table Declarations

Most basic element: an attribute and its type.

Common Types:

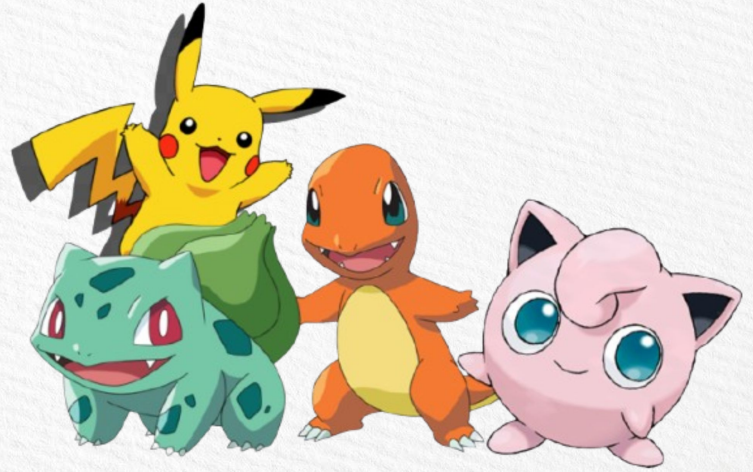
- INT or INTEGER (synonyms).
 - Also: SMALLINT and BIGINT
- REAL and DOUBLE PRECISION (FLOAT is a synonym for both).
- NUMERIC allows for arbitrary precision numbers
- CHAR(n) = fixed-length string of n characters.
- VARCHAR(n) = variable-length string of up to n characters.

Additional data type found here:

<https://www.postgresql.org/docs/current/datatype.html>

Example: Create Table

```
CREATE TABLE Pokemon (  
    name CHAR(50),  
    type_1 VARCHAR(20),  
    type_2 VARCHAR(20),  
    level INTEGER  
);
```



SQL Values

Integers and reals are represented as you would expect.

Strings are too, except they require single quotes.

- Two single quotes = real quote
 - e.g. 'Joe''s Pokemon'

SQL Values

Any value can be NULL

- Unless attribute has NOT NULL constraint
 - e.g. price REAL not null

Declaring Keys

- An attribute or list of attributes may be declared PRIMARY KEY or UNIQUE.
- Either says that no two tuples of the relation may agree in all the attribute(s) on the list.
- **Caution:** UNIQUE allows null values whereas PRIMARY KEY does not.

Example: Keys

```
CREATE TABLE Pokemon (  
    pokemon_id SERIAL PRIMARY KEY,  
    name CHAR(50),  
    type_1 VARCHAR(20),  
    type_2 VARCHAR(20),  
    level INTEGER  
);
```



Example: Keys Version 2

```
CREATE TABLE Pokemon (  
    name CHAR(50) PRIMARY KEY,  
    type_1 VARCHAR(20),  
    type_2 VARCHAR(20),  
    level INTEGER  
);
```



Example: Keys Version 3

```
CREATE TABLE Pokemon (  
    name CHAR(50) UNIQUE NOT NULL,  
    type_1 VARCHAR(20,  
    type_2 VARCHAR(20),  
    level INTEGER  
);
```



Example

Pokemon (name, species, primary_type, secondary_type)

Trainers (name, hometown, trainer_id)

Gyms (name, city, badge)

Likes (trainer, pokemon)

Owns (trainer, pokemon, nickname)

Battles (trainer, gym, battle_date)

Sells (pokemart, item, price)

Underline = **key** (tuples cannot have the same value in all key attributes)

Declaring Multi-Attribute Keys

- A key declaration can also be another element in the list of elements of a CREATE TABLE statement.
- This form is essential if the key consists of more than one attribute.
- Could be used for one-attribute keys.

Example: Multiattribute Keys

```
CREATE TABLE Teaches (  
    pokemon      CHAR(20) ,  
    move         VARCHAR(20) ,  
    level        INTEGER ,  
    PRIMARY KEY (pokemon, move)  
);
```



Primary vs. Unique

- There can be only one PRIMARY KEY for a relation, but several UNIQUE (or UNIQUE NOT NULL) attributes.
- No attribute of a PRIMARY KEY can ever be NULL in any tuple. But attributes declared UNIQUE may have NULL's, and there may be several tuples with NULL.

Constraints

- Keys
- Foreign-Key
- Attribute Checks
- ... (others next week)

Foreign Keys

A foreign key is an attribute in one table that refers to the **primary key** of another table.

This lets us connect related tables while making sure the referenced value actually exists.

Foreign Keys Example

Pokemon(name, species, primary_type)

Teaches(pokemon, move, level)

In Teaches, **pokemon** is a *foreign key* that refers to **name** in Pokemon.

This means we cannot say that "Pikachu" learns "Thunderbolt" unless "Pikachu" already appears in the Pokemon table.

Foreign Keys Example

```
CREATE TABLE Pokemon (  
    name VARCHAR(20) PRIMARY KEY,  
    species VARCHAR(20),  
    primary_type VARCHAR(20)  
);
```

```
CREATE TABLE Teaches (  
    pokemon VARCHAR(20),  
    move VARCHAR(20),  
    level INTEGER,  
    PRIMARY KEY (pokemon, move),  
    FOREIGN KEY (pokemon) REFERENCES Pokemon(name)  
);
```

The pokemon name needs to exist in the Pokemon table!



Attribute Based Checks

Constraints on the value of a particular attribute.

Add CHECK(<condition>) to the declaration for the attribute.

Attribute Based Checks

```
CREATE TABLE pokemon (  
    pokemon_id SERIAL PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    type_1 VARCHAR(20) NOT NULL,  
    type_2 VARCHAR(20),  
    level INTEGER CHECK (level >= 1 AND level <= 100),  
    hp INTEGER CHECK (hp >= 0),  
    caught_date DATE DEFAULT CURRENT_DATE  
);
```

Next Week

More DDL practice, and hierarchy relations!

Upcoming Deadlines

- PCRS
 - Week 2 practice/prepare this Sunday at 11:59 PM)
 - Week 3 prepare next Tuesday by 11:59 PM
- Checkpoint 1
 - This Sunday at 5 PM
 - Go to tutorial for DDL help!

Happy long weekend!

